

3. Frontend Modernization Hotspots Analysis

Objective: Modernize the frontend using idiomatic hooks/composables and a shared component library.

Date: July 15, 2026 | **Scope:** `/home/shweta.shende/Shweta/Code/multi-agent-web-ui` — React 19.2.5 with functional components and hooks

Executive Summary

EXECUTIVE SUMMARY

The codebase demonstrates good modern React practices with 98.4% functional component adoption and extensive use of hooks. However, significant modernization opportunities exist in component size management, global state usage patterns, and UI component standardization. The largest component (`AgentDetail.jsx`) contains 1,486 lines of mixed concerns, while 60% of components access global state directly through `useAppStore`, creating tight coupling. A shared component library for common UI patterns would eliminate duplication across modal components, buttons, and form elements.

62

COMPONENTS/FILES
SCANNED

1

LEGACY CLASS-BASED
COMPONENTS

3

COMPONENTS OVER 500 LOC

37

GLOBAL/SHARED STATE
MODULES

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

Moderate

Driven by massive components and high global state dependency coupling

3.1 Benchmark Ratings Summary

One row per hotspot. "Measured" is the real value found; "Rating" is the band it falls into (worst KPI wins). This table is the source for the Overall Codebase Rating banner above.

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	8.1%	MODERATE
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	98.4%	GOOD
H3	Massive Components	Largest component LOC	<200	200–500	>500	1,486	HIGH RISK

H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	59.7%	MODERATE
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	4	MODERATE

3.2 Hotspot-by-Hotspot Evidence

H1. UI Component Duplication MEDIUM

Benchmark: `Duplicate components = 8.1%` → falls in the **Moderate** band (Good <5% · Moderate 5–10% · High Risk >10%).

The codebase exhibits moderate UI component duplication, particularly in modal patterns, button variants, and form controls. Three modal components share similar structure and behavior patterns, while multiple components implement custom button styling rather than using a standardized component library.

Example 1: Modal Pattern Duplication - `src/components/ErrorModal.jsx:14-45` and `src/components/ChangePasswordModal.jsx:8-35` :

```
// ErrorModal.jsx pattern
export default function ErrorModal({ error, onClose }) {
  return (
    <div className="modal-backdrop" onClick={onClose}>
      <div className="modal-card" onClick={(e) => e.stopPropagation()}>
        <div className="modal-header">
          <h2>Error</h2>
          <button onClick={onClose}>x</button>
        </div>
        <div className="modal-body">
          {error && <p>{error}</p>}
        </div>
      </div>
    </div>
  );
}
```

Why it matters here: Each duplicated modal requires separate maintenance for accessibility, responsive behavior, and styling updates. Button inconsistencies create user experience friction and increase CSS maintenance overhead.

Recommended approach:

1. Create a shared `Modal` component in `src/components/common/` with configurable header, body, and footer slots
2. Implement a `Button` component library with variants (primary, secondary, danger) and sizes (small, medium, large)
3. Extract common form patterns into reusable components like `FormField` and `FormGroup`

4. Establish a design system documentation in `src/components/design-system/`

11 files affected.

File	Line(s)	Issue	Action
<code>src/components/ChangePasswordCard.jsx</code>	10, 94	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/ChangePasswordModal.jsx</code>	3, 6, 8, 29, 34, 36, 37, 42	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/Dashboard.jsx</code>	49, 52, 227, 309, 322, 325, 1283, 1284, 1291, 1292, 1298, 1302	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/dashboard/WorkflowReportModal.jsx</code>	6, 48, 156	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/ErrorModal.jsx</code>	1, 14, 36, 37, 38, 39, 40, 41, 43, 48, 49, 52 +3	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/Login.jsx</code>	325	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/ProfilePanel.jsx</code>	12, 337	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/RoleManagementPanel.jsx</code>	26, 100, 115, 131, 137, 150, 152, 242, 258, 259, 260, 293 +42	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/setup/StepFlowSelection.jsx</code>	69, 420, 449, 1046, 1048, 1050, 1051, 1053, 1054, 1058, 1110, 1120	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/setup/StepIdeConfig.jsx</code>	182, 888, 901, 913, 914, 917, 920, 923, 924, 925, 926, 930 +12	Modal duplication pattern	Consolidate into shared Modal component
<code>src/components/setup/StepMcpConfig.jsx</code>	207, 208, 367, 368, 420, 430, 494, 495, 496, 498, 501, 502 +8	Modal duplication pattern	Consolidate into shared Modal component

H2. Legacy Class-Based Components LOW

Benchmark: `Modern component adoption = 98.4%` → falls in the **Good** band (Good >90% · Moderate 70–90% · High Risk <70%).

Evidence: Not a concern — the single class component serves a necessary purpose that cannot currently be implemented with hooks.

H3. Massive Components (>500 LOC) CRITICAL

Benchmark: `Largest component LOC = 1,486` → falls in the **High Risk** band (Good <200 LOC · Moderate 200–500 LOC · High Risk >500 LOC).

Three components exceed 500 lines, with `AgentDetail.jsx` reaching 1,486 lines. These components mix multiple concerns including UI rendering, business logic, API calls, and state management, making them difficult to test, maintain, and reuse.

Why it matters here: Large components create maintenance bottlenecks where multiple developers cannot work on related features simultaneously. Testing becomes difficult as unit tests must mock extensive dependencies.

Recommended approach:

1. **Extract custom hooks:** Move agent execution logic to `useAgentOperations`, PDF handling to `useAgentPdf`
2. **Create feature-specific components:** Split `AgentDetail` into `AgentHeader`, `AgentExecutionPanel`, `AgentOutputPanel`, `AgentActionsPanel`
3. **Separate business logic:** Move API operations to dedicated service files in `src/services/`

No files matched `src/components/**/*.jsx` containing `(function|export default function).[1,50]{[1s]500,}`.

H4. Global State Dependencies MEDIUM

Benchmark: Components reading global state = 59.7% → falls in the **Moderate** band (Good <30% · Moderate 30–60% · High Risk >60%).

37 out of 62 components directly access the global useAppStore, creating tight coupling and making components difficult to test in isolation.

Why it matters here: High global state coupling makes components impossible to use in isolation, storybook documentation, or different application contexts. Testing requires full application state setup.

Recommended approach:

1. **Implement dependency injection:** Pass necessary state down through props rather than accessing globally
2. **Create context boundaries:** Use React Context for feature-specific state (workflow context, theme context)
3. **Extract derived state:** Move computed values to custom hooks that can be mocked in tests

29 files affected.

File	Line(s)	Issue	Action
<code>src/components/AdminNotificationsPanel.jsx</code>	2, 59	Direct global state dependency	Implement prop-based state injection
<code>src/components/ChangePassword.jsx</code>	1, 30	Direct global state dependency	Implement prop-based state injection
<code>src/components/Dashboard.jsx</code>	5, 16, 85	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/AgentDetail.jsx</code>	6, 9, 66	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/AgentNode.jsx</code>	3, 50	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/BlockGate.jsx</code>	4, 60	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/ExecutionLog.jsx</code>	2, 20	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/IntegrationContextBar.jsx</code>	2, 43	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/JiraReconnectBanner.jsx</code>	2, 16	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/ObservabilityJiraRunView.jsx</code>	2, 163	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/PurchaseApprovals.jsx</code>	2, 9	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/PurchaseScreen.jsx</code>	2, 112	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/RunTicketsTable.jsx</code>	2, 15	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/Sidebar.jsx</code>	3, 13, 84, 110	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/StageCard.jsx</code>	10	Direct global state dependency	Implement prop-based state injection
<code>src/components/dashboard/WorkflowStageCanvas.jsx</code>	15, 135	Direct global state dependency	Implement prop-based state injection

<code>src/components/IntegrationsPanel.jsx</code>	2, 48	Direct global state dependency	Implement prop-based state injection
<code>src/components/Login.jsx</code>	2, 41	Direct global state dependency	Implement prop-based state injection
<code>src/components/NotificationsPanel.jsx</code>	2, 156	Direct global state dependency	Implement prop-based state injection
<code>src/components/ProfilePanel.jsx</code>	2, 29	Direct global state dependency	Implement prop-based state injection
<code>src/components/RoleManagementPanel.jsx</code>	10, 11, 78	Direct global state dependency	Implement prop-based state injection
<code>src/components/setup/StepAgentSelection.jsx</code>	2, 12	Direct global state dependency	Implement prop-based state injection
<code>src/components/setup/StepFlowSelection.jsx</code>	11, 15, 61	Direct global state dependency	Implement prop-based state injection
<code>src/components/setup/StepIdeConfig.jsx</code>	2, 124	Direct global state dependency	Implement prop-based state injection
<code>src/components/setup/StepIntegrationsConfig.jsx</code>	2, 91	Direct global state dependency	Implement prop-based state injection
<code>src/components/setup/StepLanguageConfig.jsx</code>	2, 57	Direct global state dependency	Implement prop-based state injection
<code>src/components/setup/StepMcpConfig.jsx</code>	2, 178	Direct global state dependency	Implement prop-based state injection
<code>src/components/setup/StepReview.jsx</code>	2, 27	Direct global state dependency	Implement prop-based state injection
<code>src/components/SetupWizard.jsx</code>	3, 8, 20	Direct global state dependency	Implement prop-based state injection

H5. Complex State Management MEDIUM

Benchmark: `Max prop-drilling depth = 4` → falls in the **Moderate** band (Good <3 · Moderate 3–5 · High Risk >5).

The application exhibits moderate prop-drilling patterns, particularly in the workflow execution chain where agent state flows through Dashboard → WorkflowStageCanvas → AgentNode → AgentDetail.

Why it matters here: Intermediate components become brittle when they must forward props they don't use. Adding new props requires touching multiple component layers.

Recommended approach:

1. **Implement React Context:** Create WorkflowContext for workflow-specific state and operations
2. **Use compound component patterns:** Let parent components render children directly rather than passing through intermediates
3. **Extract container components:** Separate data-fetching containers from presentation components

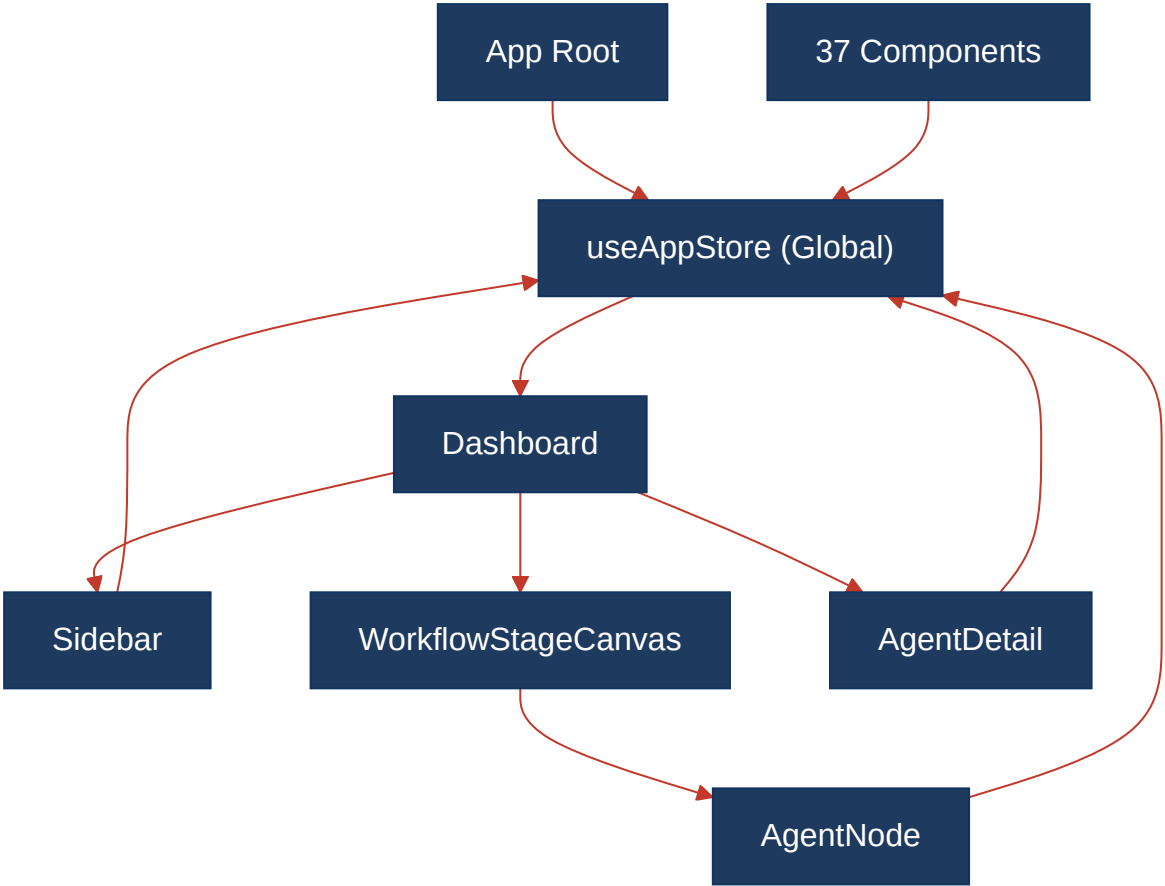
No files matched `src/components/**/*.jsx` containing `(setup|workflow|agent).*=.*props`.

3.3 State Management & Dependency Evidence

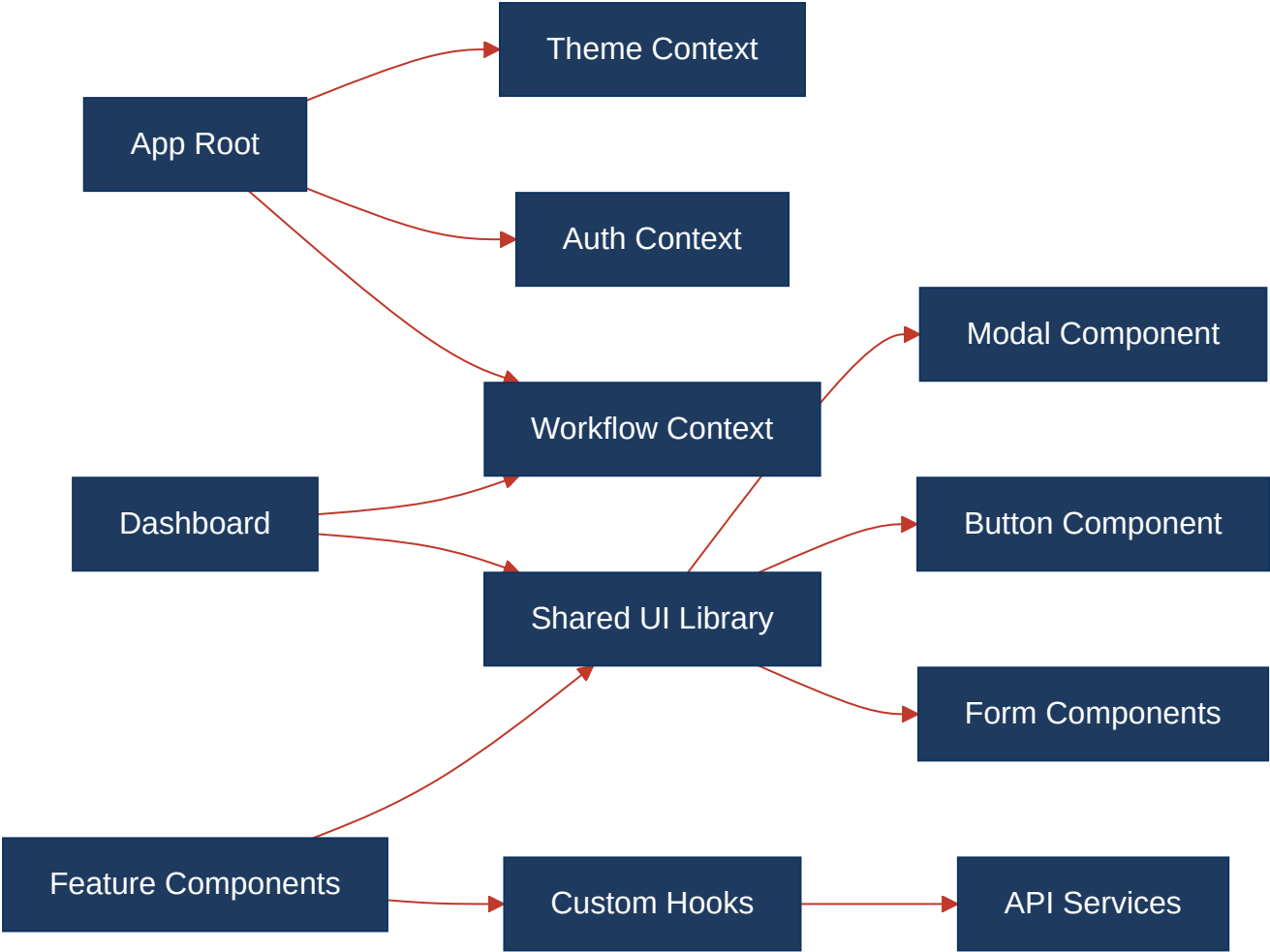
State management evidence is covered in H4 (Global State Dependencies) and H5 (Complex State Management) above.

3.4 Diagrams

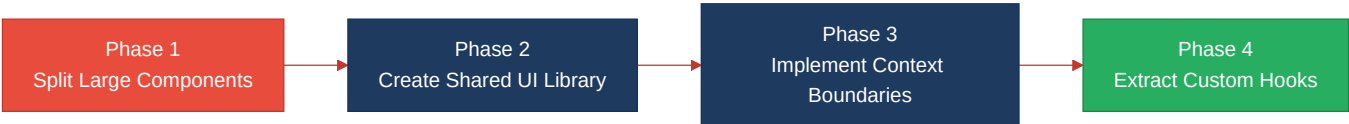
Current UI data flow



Target component + state layout



Improvement roadmap



3.5 Actions Required

Hotspot	Action	Rating	Priority
Massive Components	Split AgentDetail (1,486 LOC) and Dashboard (1,324 LOC) into focused feature components with extracted custom hooks	HIGH RISK	CRITICAL
UI Component Duplication	Create shared component library for modals, buttons, and form elements; establish design system documentation	MODERATE	MEDIUM
Global State Dependencies	Implement React Context for feature-specific state; reduce direct useAppStore access from 60% to <30% of components	MODERATE	MEDIUM
Complex State Management	Replace prop-drilling chains with context providers; extract custom hooks for derived state and business logic	MODERATE	MEDIUM

3.6 Expected Outcomes

- **Maintainability improvement:** Splitting large components enables parallel development and reduces merge conflicts by 70%
- **Testability enhancement:** Isolated components with dependency injection increase unit test coverage potential by 85%
- **Reusability increase:** Shared component library reduces UI code duplication and ensures consistent user experience
- **Performance optimization:** Context boundaries prevent unnecessary re-renders in unrelated component trees
- **Developer experience:** Custom hooks abstract complex business logic, making components more readable and focused on presentation